

Utilizando Julia para Calcular e Visualizar Superfícies

Osmar Cardoso Lopes Filho

osmarclopesfilho@gmail.com

[git link]

1 Instalação

1.1 Julia

Para instalar a linguagem Julia, acesse o [site oficial](#) e siga as instruções na aba “Download”. A partir disso, você terá em seu sistema dois executáveis em CLI (*command line interface*): `julia` e `juliaup`. Seus usos são:

- `julia`: Inicializar o REPL (Read-Eval-Print-Loop) interativo da linguagem ou compilar código em Julia caso um arquivo seja passado junto com o comando;
- `juliaup`: Atualizar e gerenciar versões da linguagem Julia no sistema,

1.2 Kernel IJulia

Para utilizar Julia através do Jupyter Notebook ou Lab, instale primeiro a linguagem e depois siga as instruções de instalação do pacote `IJulia.jl`. Feito isso, o Jupyter deve ser capaz de identificar o *kernel* de Julia instalado pelo pacote em seu sistema.

2 Renderizando Superfícies Paramétricas

Para renderizar computacionalmente superfícies, precisamos descrevê-las através de uma parametrização. Isso nos permite representar de forma aproximada, limitados pelas capacidades físicas do computador, a superfície em questão. Afinal, estamos lidando com objetos contínuos e, portanto, impossíveis de serem discretizados de forma completa.

O foco desse trabalho é posto sobre superfícies em $\mathbb{R}^3$ com parâmetros em $\mathbb{R}^2$. Lidaremos, então, apenas com parametrizações da forma $f : U \subset \mathbb{R}^2 \rightarrow \mathbb{R}^3$. Além disso, consideraremos as condições de regularidade da superfície apenas em um momento posterior, priorizando agora a construção e visualização de alguns de seus pontos.

Dada uma função parametrizadora $f : U \subset \mathbb{R}^2 \rightarrow \mathbb{R}^3$, precisamos tomar alguns pontos em U para visualizarmos computacionalmente a superfície. A maneira mais simples de realizar isso é tomando pontos em uma treliça retangular, como ilustrado a seguir:

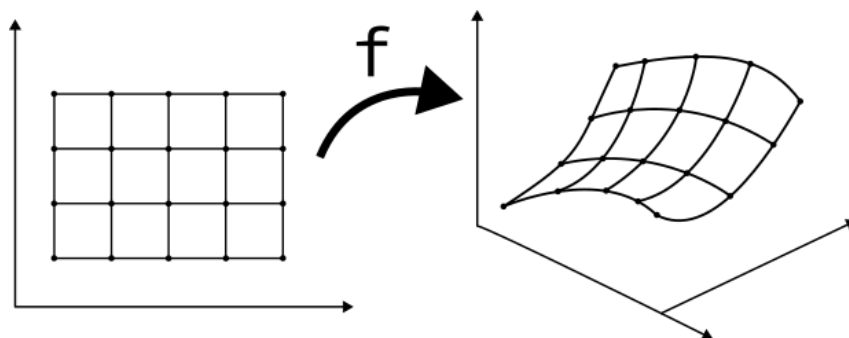


Figura 1: Pontos em uma treliça retangular.

Contudo, isso trás um empecilho: Como somos apenas capazes de renderizar computacionalmente triângulos, precisamos que nossa superfície seja aproximada por uma malha triangular. Isso não é obtido diretamente através da treliça retangular. De fato, para obtermos essa malha, precisamos cortar cada quadrilátero em dois triângulos, o que pode afetar negativamente a visualização uma vez que os quadriláteros podem não ser planos.

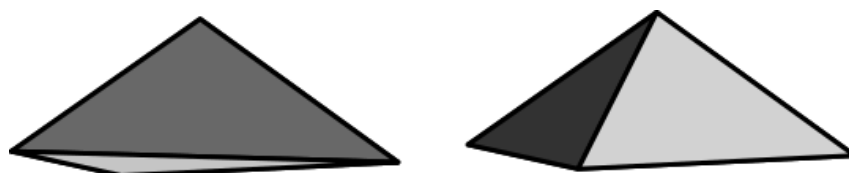


Figura 2: Duas formas de separar um quadrilátero em triângulos. Note que elas aproximam curvaturas diferentes.

Devido a isso, escolhemos selecionar pontos a partir de uma treliça triangular equilátera:

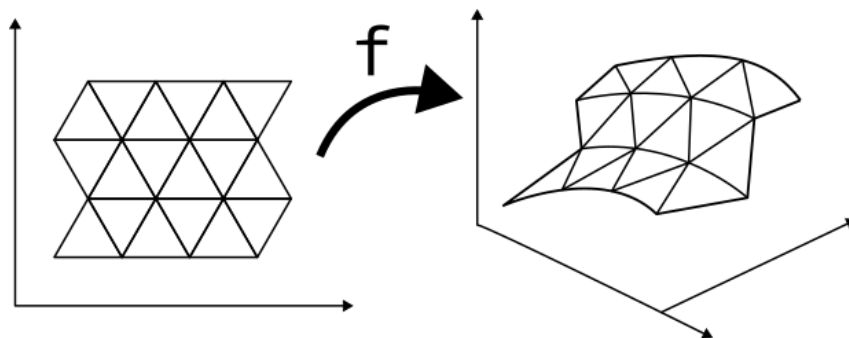


Figura 3: Pontos em uma treliça triangular.

O primeiro passo é construir uma seleção de pontos com as devidas coordenadas. Armazenaremos essa informação na forma de uma array tridimensional onde a primeira e a segunda dimensão representam a posição do ponto na treliça e ao longo da terceira guardamos as coordenadas do ponto no espaço. A posição do ponto na treliça é determinada da seguinte forma:

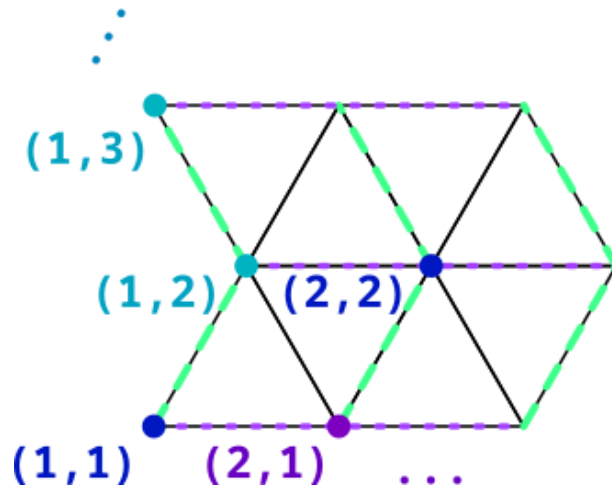


Figura 4: Posição dos pontos na treliça. As coordenadas são os índices da primeira e segunda dimensão da array representativa da treliça.

A nossa função responsável por criar essa array é a `trigvertx_rect`, a qual recebe dimensões de um retângulo e a largura dos lados dos triângulos e retorna uma treliça contida num retângulo com as dimensões passadas e ponto inferior esquerdo na origem.

Agora que possuímos uma estrutura para representar nossos pontos, precisamos determinar quais deles formam triângulos uns com os outros. Essa informação será usada pelo pacote `Plots.jl` para renderizar uma aproximação da curva.

Seja $m = y \bmod 2$, definimos os vizinhos do ponto (x, y) , i.e. os outros pontos com os quais ele forma triângulos, da seguinte forma:

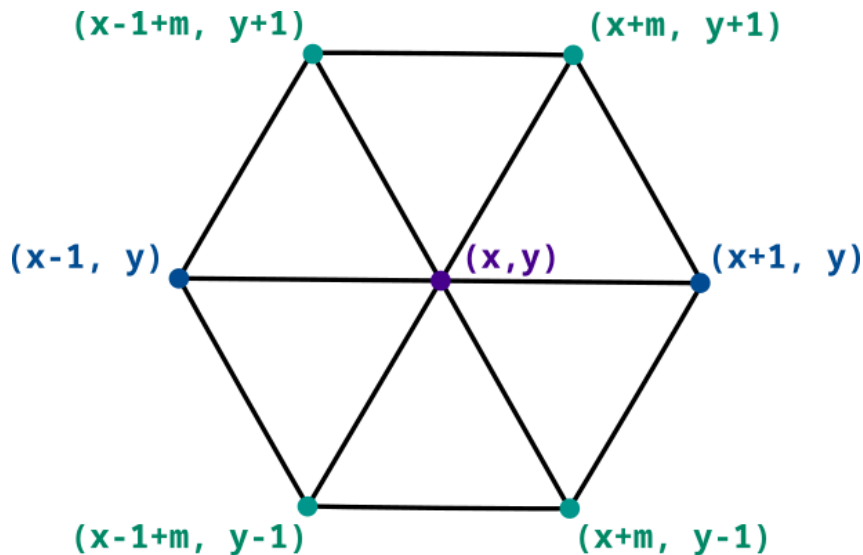


Figura 5: Vizinhos do ponto (x, y) na treliça.

A nossa função responsável por determinar as conexões entre os pontos é a `trimesh_rect`, a qual recebe a saída da `trigvertx_rect` e retorna uma array bidimensional onde cada linha possui 3 identificadores de pontos que compõem 1 triângulo.

Agora, a partir da informação que possuímos, podemos aplicar a função f sobre cada ponto e renderizar o mesh resultante. A nossa função `apply_trigvertx` recebe o resultado `trigvertx_rect` e uma função qualquer que receba 2 argumentos numéricos e retorne 3. Seu

retorno tem o mesmo formato da `trigvertx_rect`, mas com 3 valores na terceira dimensão da array. Essa função qualquer recebida será a representação computacional da f .

Agora, a seguir, renderizamos uma esfera:

```
# Adicionamos  $\pi/10$  (a largura dos lados do triângulo) para repetir a primeira
coluna de pontos.
# Efetivamente, isso conecta os últimos pontos aos primeiros.
vertx = trigvertx_rect( $2\pi+(\pi/10)$ ,  $\pi+(\pi/10)$ ,  $\pi/10$ )

# Subtraímos 1 de todos os valores para converter a indexação em 1 para uma
indexação em 0,
# requerida pela função mesh3d.
conns = trigmesh_rect(vertx).-1

f = ( $\theta$ ,  $\phi$ ) -> (sin( $\phi$ )*cos( $\theta$ ), sin( $\phi$ )*sin( $\theta$ ), cos( $\phi$ ))
vertx3d = apply_trigvertx(vertx, f1)

# Convertemos vertx3d em uma array onde cada linha possui as coordenadas no
espaço de 1 ponto.
# Efetivamente, estamos planificando as duas primeiras dimensões de vertex3d.
A maneira com a qual
# isso é feito é idêntica à que trigmesh_rect segue. Isso garante que os
identificadores em conns
# estejam corretos.
meshpoints = [vec(vertx3d[:, :, 1]) vec(vertx3d[:, :, 2]) vec(vertx3d[:, :, 3])]

mesh3d(
  meshpoints[:, 1], meshpoints[:, 2], meshpoints[:, 3];
  connections=(conns[:, 1], conns[:, 2], conns[:, 3]),
  linewidth=0.2, linecolor=:black, linealpha=1,
  fillalpha=0,
  camera=(40, -20),
  showaxis=false,
  grid=false,
  legend=:none
)
```

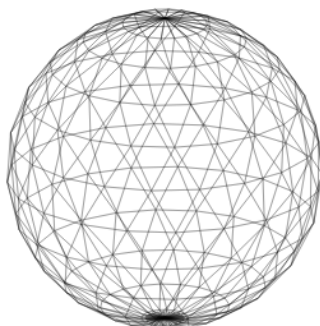


Figura 6: Imagem resultante.

3 Computando Formas

Além de visualizar aproximações das curvas, computaremos também a primeira e segunda forma fundamental. Para isso, utilizaremos o pacote `Symbolics.jl`.

Para obtermos a primeira forma, precisamos das derivadas parciais da nossa curva parametrizada. A partir desse ponto, será necessário considerar as condições de regularidade da curva. Dada uma parametrização $f : U \subset \mathbb{R}^2 \rightarrow \mathbb{R}^3$, dizemos que a curva é regular se:

- f é diferenciável de classe C^∞
- $\forall p \in U, dX_q : \mathbb{R}^2 \rightarrow \mathbb{R}^3$

Onde dX_q é o plano tangente a curva no ponto p .

Consideremos então f sendo a parametrização da esfera unitária, i.e.:

$$f(\theta, \varphi) = (\sin(\varphi) \cos(\theta), \sin(\varphi) \sin(\theta), \cos(\varphi))$$

Obtemos os coeficientes da primeira forma da seguinte maneira:

```
using Symbolics, LinearAlgebra

@variables θ ϕ

(x,y,z) = (sin(ϕ)*cos(θ), sin(ϕ)*sin(θ), cos(ϕ))

Dθ = Differential(θ)
Dϕ = Differential(ϕ)

Sθ = expand_derivatives.(Dθ.([x,y,z]))
Sϕ = expand_derivatives.(Dϕ.([x,y,z]))

E = dot(Sθ, Sθ)
F = dot(Sθ, Sϕ)
G = dot(Sϕ, Sϕ)

firstform_mat = [
    E F;
    F G
]
```

Feito isso, podemos substituir θ e φ pelos valores alguma entrada e então calcular formas de vetores:

```
firstform_mat = substitute.(firstform_mat, (Dict{θ => π/2, ϕ => π/2},))
x = [1,0]
y = [0,1]

dot(x, firstform_mat, y)
```

Já para a segunda forma, prosseguimos da seguinte forma:

```

n = cross(Sθ, Sφ)/norm(cross(Sθ, Sφ))

Sθ2 = expand_derivatives.((Dθ^2).([x,y,z]))
Sθφ = expand_derivatives.((Dθ*Dφ).([x,y,z]))
Sφ2 = expand_derivatives.((Dφ^2).([x,y,z]))

L = dot(Sθ2, n)
M = dot(Sθφ, n)
N = dot(Sφ2, n)

secondform_mat = [
    L M;
    M N
]

```